Listing 12.5 Validating user input in a web form

```
def validName(name: String): Validation[String, String] =
  if (name != "") Success(name)
  else Failure("Name cannot be empty")

def validBirthdate(birthdate: String): Validation[String, Date] =
  try {
    import java.text._
    Success((new SimpleDateFormat("yyyy-MM-dd")).parse(birthdate))
  } catch {
    Failure("Birthdate must be in the form yyyy-MM-dd")
  }

def validPhone(phoneNumber: String): Validation[String, String] =
  if (phoneNumber.matches("[0-9]{10}"))
       Success(phoneNumber)
  else Failure("Phone number must be 10 digits")
```

And to validate an entire web form, we can simply lift the `WebForm` constructor with `map3`:

```
def validWebForm(name: String,
                 birthdate: String,
                 phone: String): Validation[String, WebForm] =
  map3(
    validName(name),
    validBirthdate(birthdate),
    validPhone(phone))(
    WebForm(_,_,_))
```

If any or all of the functions produce `Failure`, the whole `validWebForm` method will return all of those failures combined.

12.5 The applicative laws

This section walks through the laws for applicative functors.[4] For each of these laws, you may want to verify that they're satisfied by some of the data types we've been working with so far (an easy one to verify is `Option`).

12.5.1 Left and right identity

What sort of laws should we expect applicative functors to obey? Well, we should definitely expect them to obey the functor laws:

```
map(v)(id) == v
map(map(v)(g))(f) == map(v)(f compose g)
```

This implies some other laws for applicative functors because of how we've implemented `map` in terms of `map2` and `unit`. Recall the definition of `map`:

[4] There are various other ways of presenting the laws for `Applicative`. See the chapter notes for more information.